

WEEK 02 MODULE

Programming Practice

5. Prioritize Bug Reports (Selection Sort)

que. 2 What is Sorting?

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- | | | |
|---------------------|---|--------|
| i. Bubble Sort | } | Common |
| ii. Selection Sort | | |
| iii. Insertion Sort | | |
| iv. Merge Sort | | |
| v. Quick Sort | | |
| vi. Heap Sort | | |

Insertion Sort

Definition : Insertion Sort builds the sorted array one element at a time by taking an element from the unsorted portion and inserting it into its correct position within the sorted portion.

Key Points:-

- Similar to arranging playing cards in your hand.
- Maintains a sorted portion on the left side.
- Efficient for small or nearly sorted datasets.
- Stable sorting algorithm.
- Requires fewer comparisons when data is nearly sorted.

How It Works:-

Array: 5 3 8 1

Assume first element is already sorted. [5] 3 8 1	Step 1 Take 3. Compare with 5. $3 < 5$ Shift 5 right. Insert 3. 3 5 8 1	Step 2 Take 8. Compare with 5. $8 > 5$ Insert directly. 3 5 8 1	Step 3 Take 1. Compare with 8. Shift 8. Compare with 5. Shift 5. Compare with 3. Shift 3. Insert 1. 1 3 5 8 Array sorted.
--	--	---	--

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

1. Problem Analysis

A software development team receives multiple bug reports with different priority values. These bug reports may not be stored in any particular order. To ensure efficient issue tracking and resolution, the bug reports need to be arranged according to their priority levels.

The challenge is to organize the priorities in ascending order so that higher-priority issues can be identified and managed systematically.

2. Algorithm Selection Analysis

Selection Sort is chosen because it repeatedly selects the smallest element from the unsorted portion of the array and places it at its correct position.

Reasons for selecting Selection Sort:

- Simple and easy to implement.
- Performs sorting through selection rather than repeated swapping.
- Reduces the number of swaps compared to Bubble Sort.
- Suitable for understanding the concept of selecting and positioning elements.

The algorithm gradually builds a sorted portion at the beginning of the array while reducing the size of the unsorted portion.

3. Core Idea / Logic

Selection Sort divides the array into two regions:

✦ Sorted Portion

Contains elements that are already placed in their correct positions.

✦ Unsorted Portion

Contains elements that still need to be examined.

For each pass:

- Assume the first unsorted element is the smallest.
- Search the remaining unsorted portion.
- Identify the actual smallest element.
- Swap it with the first unsorted element.
- Expand the sorted portion by one position.

This process continues until all elements are arranged in ascending order.

4. Execution Flow

Input

- Read the number of bug reports (n).
- Store all priority values in an array.

Processing

- Traverse the array.
- Find the smallest element in the unsorted portion.
- Swap it with the first unsorted element.
- Repeat until all elements are placed in their correct positions.

Output

- Display the sorted bug priorities in ascending order.

Step-by-Step Program Execution

Using sample input:

5

9 2 8 1 5

Step 1

Read the number of bug reports.

User enters 5.

```
int n = sc.nextInt();
```

n=5

Step 2

Create an array of size n

```
int arr[] = new int[n];
```

Step 3

Accept all priority values from the user and store them in the array.

9 2 8 1 5

Array becomes:

Index : 0 1 2 3 4

Value : 9 2 8 1 5

Step 4

Assume the first unsorted element is the minimum element.

Step 5

Traverse the remaining unsorted portion to find the actual minimum value.

Step 6

Update the minimum index whenever a smaller element is found.

Step 7

After completing the search, swap the minimum element with the first unsorted element.

Step 8

Expand the sorted portion by one position.

Step 9

Repeat the process for the remaining unsorted elements.

Step 10

Display the sorted bug priorities.

5. Key Observations

- The array does not need to be initially sorted.
- During each pass, the smallest element from the unsorted portion is selected.
- Only one swap is performed per pass.
- The sorted portion grows from left to right.
- The unsorted portion becomes smaller after each iteration.
- Best-case and worst-case scenarios perform a similar number of comparisons.
- Selection Sort generally performs fewer swaps than Bubble Sort.
- The algorithm sorts the array in-place without requiring an additional array.
- Every element eventually reaches its correct position through repeated selection of minimum values.

6. Program Structure Analysis

i. Variable n

```
int n;
```

Purpose:

- Stores the total number of bug priorities.
- Determines the size of the array.
- Controls input and sorting operations.

ii. Array arr[]

```
int arr[] = new int[n];
```

Purpose:

- Stores all bug priority values.
- Holds both unsorted and sorted data during execution.
- Allows indexed access for searching and swapping.

iii. Input Loop

```
for(int i = 0; i < n; i++)  
{  
    arr[i] = sc.nextInt();  
}
```

Purpose:

- Accepts bug priority values from the user.
- Stores them in the array before sorting begins.

iv. Outer Loop

```
for(int i = 0; i < n - 1; i++)
```

Purpose:

- Controls the number of sorting passes.
- Identifies the position where the next smallest element should be placed.
- Gradually expands the sorted portion of the array.

v. Variable minIndex

```
int minIndex = i;
```

Purpose:

- Assumes the first unsorted element is the smallest.
- Stores the index of the minimum element found during the current pass.
- Gets updated whenever a smaller element is discovered.

vi. Inner Loop

```
for(int j = i + 1; j < n; j++)
```

Purpose:

- Traverses the unsorted portion of the array.
- Searches for the actual minimum element.
- Compares all remaining elements with the current minimum.

vii. Comparison Condition

```
if(arr[j] < arr[minIndex])
```

Purpose:

- Checks whether a smaller element exists.
- Updates minIndex whenever a new minimum value is found.

viii. Minimum Index Update

```
minIndex = j;
```

Purpose:

- Records the position of the newly discovered minimum element.
- Ensures the smallest value can be placed correctly after the pass is completed.

ix. Temporary Variable

```
int temp;
```

Purpose:

- Temporarily stores one value during swapping.
- Prevents data loss while exchanging elements.

x. Swapping Process

```
temp = arr[i];  
arr[i] = arr[minIndex];  
arr[minIndex] = temp;
```

Purpose:

- Places the smallest element at its correct position.
- Expands the sorted portion by one element.
- Moves the selected minimum value from the unsorted region to the sorted region.

xi. Output Loop

```
for(int i = 0; i < n; i++)
```

Purpose:

- Traverses the sorted array.
- Displays bug priorities in ascending order.

Why do we use minIndex = i?

```
int minIndex = i;
```

At the beginning of every pass, Selection Sort assumes:

"The first element of the unsorted portion is the smallest."

Example: 40 20 10 30

Pass 1:

40 20 10 30

↑

minIndex

After checking all remaining elements:

40 20 10 30

↑

Actual Minimum

So:

```
minIndex = 2;
```

and then the swap occurs:

10 20 40 30

This process repeats until the entire array becomes sorted.

7. Program

```
package week2_module;
import java.util.Scanner;
public class PrioritizeBugReportsSelectionSort {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input number of bug reports
        System.out.print("Enter the number of bug reports: ");
        int n = sc.nextInt();

        // Create array
        int arr[] = new int[n];

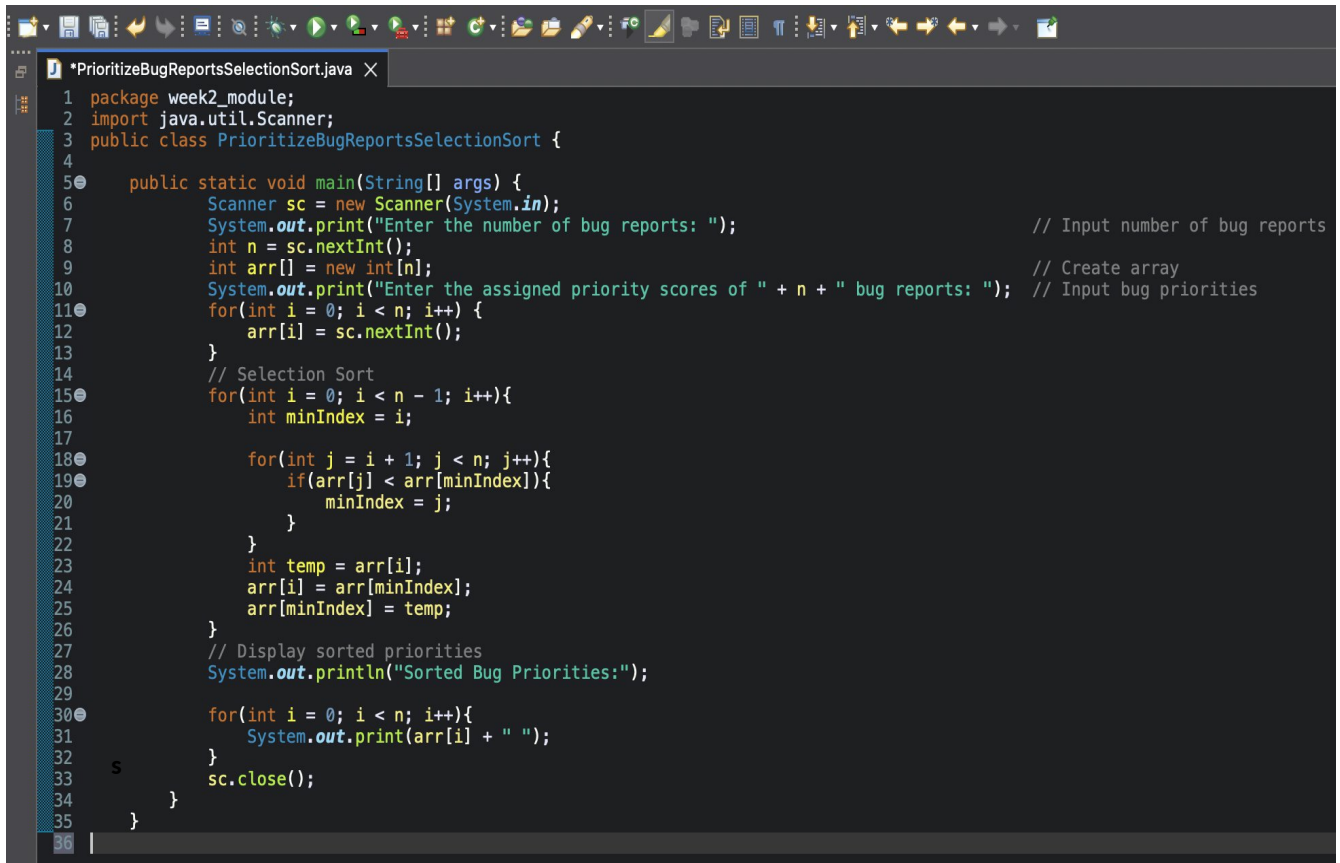
        // Input bug priorities
        System.out.print("Enter the assigns priority scores of " + n + " bug reports:");
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        // Selection Sort
        for(int i = 0; i < n - 1; i++){
            int minIndex = i;
            for(int j = i + 1; j < n; j++){
                if(arr[j] < arr[minIndex]){
                    minIndex = j;
                }
            }
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
        // Display sorted priorities
        System.out.println("Sorted Bug Priorities:");
        for(int i = 0; i < n; i++){
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

8. Program's output

```
Enter the number of bug reports: 5
Enter the assigned priority scores of 5 bug reports: 9
2
8
1
5
Sorted Bug Priorities:
1 2 5 8 9
```

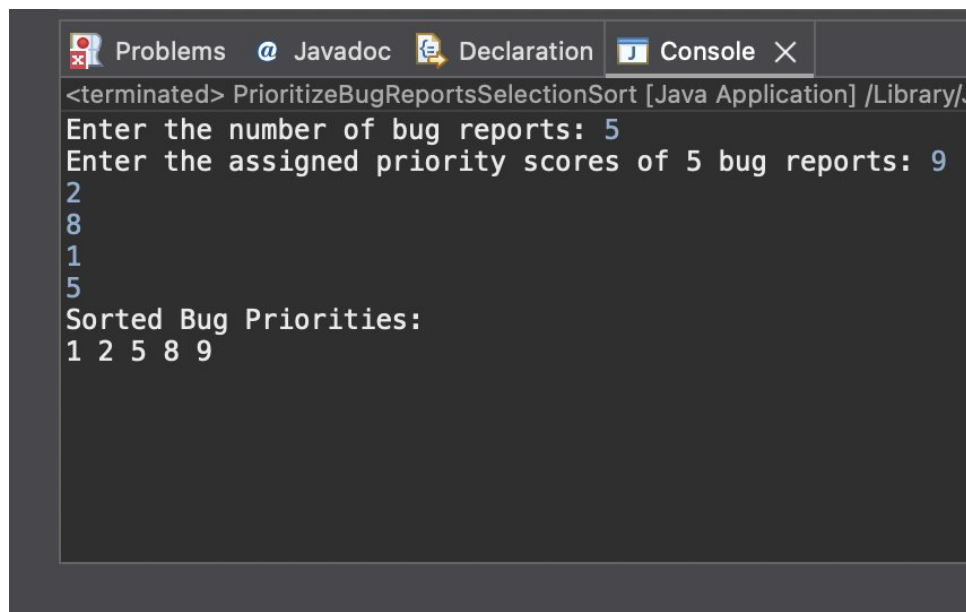

9. Screenshot

- program



```
1 package week2_module;
2 import java.util.Scanner;
3 public class PrioritizeBugReportsSelectionSort {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         System.out.print("Enter the number of bug reports: "); // Input number of bug reports
8         int n = sc.nextInt();
9         int arr[] = new int[n]; // Create array
10        System.out.print("Enter the assigned priority scores of " + n + " bug reports: "); // Input bug priorities
11        for(int i = 0; i < n; i++) {
12            arr[i] = sc.nextInt();
13        }
14        // Selection Sort
15        for(int i = 0; i < n - 1; i++){
16            int minIndex = i;
17
18            for(int j = i + 1; j < n; j++){
19                if(arr[j] < arr[minIndex]){
20                    minIndex = j;
21                }
22            }
23            int temp = arr[i];
24            arr[i] = arr[minIndex];
25            arr[minIndex] = temp;
26        }
27        // Display sorted priorities
28        System.out.println("Sorted Bug Priorities:");
29
30        for(int i = 0; i < n; i++){
31            System.out.print(arr[i] + " ");
32        }
33        sc.close();
34    }
35 }
```

- program's output



```
<terminated> PrioritizeBugReportsSelectionSort [Java Application] /Library/
Enter the number of bug reports: 5
Enter the assigned priority scores of 5 bug reports: 9
2
8
1
5
Sorted Bug Priorities:
1 2 5 8 9
```